

Contents

Reproducibility and Determinism in LLM-Generated vs. Rule-Based Spatial Query Plans: A Vienna Accessibility Case Study	1
Abstract	1
1. Introduction	2
2. Related Work	3
3. Case Study Design	4
4. Methods	5
5. Results	6
6. Discussion	12
7. Limitations	13
8. Conclusion and Future Work	14
9. Reproducibility and Data & Code Availability	15
References	15

Reproducibility and Determinism in LLM-Generated vs. Rule-Based Spatial Query Plans: A Vienna Accessibility Case Study

Author: Araz Shah Karami

Repository: github.com/arazshah/smart-spatial-vienna-accessibility — every notebook, raw run log, processed dataset, figure, and result table referenced in this paper is released here under the MIT license, together with a step-by-step reproduction guide (Section 9).

Abstract

Natural-language interfaces to spatial analysis tools increasingly rely on large language models (LLMs) to translate a question into an executable query plan. Unlike a hand-written rule-based planner, an LLM-backed planner is not deterministic by construction, which raises a practical question for anyone building on top of one: how much does its output actually vary across repeated runs of the same question, and does that variation matter for the final answer? This paper presents a small, fully reproducible case study built on [Smart Spatial System](#), an open-source spatial query framework. We pose one fixed natural-language accessibility question — rank Vienna’s 23 municipal districts by accessibility to metro stations, schools, and parks — to two planners: a deterministic rule-based planner, and an LLM-backed planner run $N = 20$ times at a fixed temperature. We compare the two arms with a three-layer metric covering structural agreement (Plan Agreement Rate), parametric agreement (the weights and distance cutoffs each plan assigns), and outcome agreement (rank stability, via Spearman’s ρ). We find that once several validation gaps in the underlying framework were fixed (documented here as a paired failure-then-resolution finding), the LLM arm was perfectly reliable across all 20 runs (PAR = 1.0, Rank Stability = 1.0) and its final ranking agreed with the rule-based arm’s almost exactly ($\rho = 0.9990$). The one point of disagreement is a modeling choice, not noise: the LLM invents a flat weighting across amenity types where the rule-based arm was given a deliberately unequal one, yet Vienna’s amenity density makes this choice invisible for all but the two lowest-

ranked districts. A smaller follow-up pilot ($N = 5$ per condition) finds this same perfect reliability recurring across two additional models reached through a different provider and four independent phrasings of the question, once a second, analogous harness gap specific to those conditions was identified and closed — though the pilot could not successfully include a smaller or weaker model, leaving that generalization open. We report the full methodology, results, and limitations, and release all code, data, and run logs.

1. Introduction

Spatial analysis tools built for end users increasingly expose a natural-language front end: a user asks a question in plain language, and an LLM translates it into a sequence of operations against a geospatial backend. This is attractive because it removes the need to learn a query language, but it introduces a property that a hand-written rule-based translator does not have — the same question, asked twice, is not guaranteed to produce the same plan.

For a one-off exploratory query this may not matter. It matters a great deal for any use case where the plan or its output is treated as an artifact of record: a report generated once and reused, a ranking published and cited, a decision made on the strength of a single run. Before an LLM-backed planner can be trusted for that kind of use, its actual run-to-run variability needs to be measured, not assumed.

This paper measures it directly, using a real spatial analysis task rather than a synthetic benchmark. We ask a single natural-language question — “rank Vienna’s districts by accessibility to metro, schools, and parks” — through [Smart Spatial System](#), a spatial query framework that supports both a deterministic rule-based query builder and an LLM-backed one (LLMQuerySpecGenerator). We run the rule-based arm as a fixed reference and the LLM arm 20 times at temperature 0.1, and compare the two along three layers: does the LLM produce the same *shape* of plan every time (structural agreement), does it choose the same weights and distance cutoffs every time (parametric agreement), and does it produce the same final district ranking every time, and does that ranking agree with the deterministic reference (outcome agreement)? The full metric is defined precisely in Section 4.3 and in the repository’s [comparison_metric.md](#).

Getting to a clean measurement was not straightforward: early attempts surfaced real defects in the underlying framework’s validation logic, which silently produced degenerate rankings rather than raising an error. We treat the discovery and resolution of those defects as part of the contribution, not as noise to be edited out of the story (Section 6.1) — it is itself a data point about where the risk in an LLM-backed pipeline actually lives. A smaller follow-up pilot (Section 5.5) then checks whether the main result’s perfect reliability is specific to the one model and one phrasing tested, or generalizes further — and finds the same harness-completeness lesson recurs before it finds a clean result.

This is a small case study — one city, one question, one model, one temperature — and we are explicit about that scope throughout (Section 7). Its purpose is to demonstrate a concrete, repeatable method for measuring an LLM planner’s reliability against a deterministic baseline, and to report what that method found for this particular system, not to make a general claim about LLM planners at large.

1.1 What Smart Spatial System does, and what this case study adds

Smart Spatial System is an open-source Python framework for answering spatial (GIS) questions over vector map layers — district boundaries, points of interest, transit stops, and the like. Given a question, it does not answer it directly; it first *plans*: it produces a `QuerySpec`, an explicit, ordered list of GIS operations (reproject coordinates, find the nearest feature of a given type, compute a weighted score, rank the results), and only then *executes* that plan against the real data to produce a final output — here, a ranked table of districts. The framework offers two different ways to produce that plan: a **rule-based builder**, which assembles the operation list mechanically from a fixed configuration supplied by the developer, and an **LLM-backed generator** (`LLMQuerySpecGenerator`), which sends the natural-language question to a large language model and asks it to produce the same kind of operation list itself, with no fixed configuration to fall back on.

This case study does not modify or extend Smart Spatial System’s functionality. It uses the framework exactly as published, asks both of its planners the identical real question — *rank Vienna’s 23 districts by accessibility to metro, schools, and parks* — and then measures something the framework itself does not measure: whether the LLM-backed planner, asked the same question repeatedly, keeps producing the same plan and the same final ranking, and how that compares to the rule-based planner, which is deterministic by definition. Sections 3–5 describe exactly how that comparison is built and what it found; Section 5.5 repeats the same measurement across several other LLMs and several rewordings of the question, to check whether the finding is specific to one model and one phrasing.

Concretely, the pipeline this paper runs (for either planner) is: **input** — Vienna’s district boundaries, metro stations, schools, and parks, as real OpenStreetMap layers (Section 3.3) — flows through a short chain of GIS **operations** (coordinate-system transforms, nearest-feature distance lookups, a weighted scoring step, a final ranking step) that the chosen planner assembled, and produces **output** — a table of all 23 districts with an accessibility score and rank. Every run of that pipeline, for every planner and every model tested, is logged in full (the exact plan, whether it succeeded, the resulting ranking, and how long it took) and committed to the repository as a raw JSON file, so every number in this paper traces back to a specific, inspectable run rather than to a summary alone (Section 9).

2. Related Work

This case study sits at the intersection of two lines of work that, to our knowledge, have not been directly connected before.

The first is *accessibility measurement in urban and regional analysis*, and specifically the recent “15-minute city” framing, which scores places by proximity to daily-need amenities such as transit, schools, and green space. Empirical studies of this kind — for example the measurement of proximity-based accessibility and inequality across Barcelona (Vich et al., *A city of cities: Measuring how 15-minutes urban accessibility shapes human mobility in Barcelona*; see also the PLOS ONE version, journals.plos.org/plosone/article?id=10.1371/journal.pone.0250080) and subsequent work on measuring 15-minute-city access and inequality more broadly (*Measuring 15-minute city access and inequality*) and on proximity-based accessibility as a design and evaluation lens (*Just around the corner: Accessibility by proximity in the 15-minute city*) — establish the administrative-unit-level, multi-

amenity scoring approach this paper adopts for Vienna’s 23 districts. We rely on this literature only for that framing; the accessibility scoring itself is a means to an end here, not the paper’s contribution, and the district-level ceiling effect we report in Section 7 is a direct consequence of choosing this framing at this spatial resolution.

The second is the emerging literature on *the reliability and reproducibility of LLM agents in multi-step, tool-calling pipelines* — the actual subject of this paper’s measurement. Work directly measuring behavioral reproducibility across repeated runs of a tool-calling pipeline ([How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines](#)) and work on the role of harness- and framework-level validation in making agentic execution predictable ([Harness Engineering for Predictable Agentic Systems: An Empirical Study of Deterministic Execution Constraints](#)) both speak to this paper’s central finding: that an LLM planner’s apparent unreliability can be a property of the surrounding framework’s validation gaps rather than the model itself, and that closing those gaps is what produces the perfect-reliability result reported in Section 5. A broader survey of how agentic systems are evaluated ([Evaluation and Benchmarking of LLM Agents: A Survey](#)) situates the three-layer metric used here (structural, parametric, outcome agreement) relative to existing agent-evaluation practice, which more commonly reports task success rate alone.

3. Case Study Design

3.1 Units of analysis

The unit of analysis is Vienna’s 23 municipal districts (*Gemeindebezirke*), by official administrative boundary. Districts were chosen over arbitrary sample points because they are a fixed, citable, small ($N = 23$) set that requires no invented sampling scheme, and district-level accessibility scoring of this kind is standard practice in the 15-minute-city literature cited above.

3.2 Amenities

Three amenity layers are scored, each with its own weight and maximum relevant distance, reflecting a deliberate modeling judgment about their relative importance to daily life:

Amenity	Weight	Max. distance
Metro (U-Bahn) stations	3.0	800 m
Schools	2.0	1,000 m
Parks	1.0	1,500 m

These values are the configuration given to the rule-based arm (Section 4.1); the LLM arm is never shown them (Section 4.2), which is what makes the two arms’ agreement or disagreement on weighting meaningful rather than circular.

3.3 Data

All three amenity layers and the district boundaries come from OpenStreetMap via the Overpass API (district boundaries: `boundary=administrative, admin_level=9`;

metro: railway=station + station=subway; schools: amenity=school; parks: leisure=park), documented in full with exact queries in the repository’s data/README.md and scripts/download_vienna_data.md, and licensed under the Open Database License with attribution retained in the processed files. The resulting dataset covers 23 districts, 109 metro stations, 211 schools, and 1,063 parks.

3.4 The two arms

- **Rule-based arm** (`build_accessibility_query_spec`): the operation chain — coordinate transform, nearest-neighbor distance to each amenity type, scoring, ranking — follows mechanically from the amenity list in Section 3.2. Deterministic by construction; run 5 times as a sanity check that it is in fact deterministic (PAR = 1.0, Rank Stability = 1.0, both trivially, since every run is byte-identical).
- **LLM-backed arm** (`LLMQuerySpecGenerator`): the same underlying question, phrased in natural language and given to an LLM with no amenity weights or distance cutoffs supplied, run $N = 20$ times at a fixed temperature of 0.1 (matching the framework’s own documented default, chosen to keep this a single-condition first pass rather than doubling the analysis surface with a temperature sweep — see Section 8).

4. Methods

4.1 Rule-based plan construction

The rule-based arm builds its plan directly from the amenity configuration in Section 3.2: a chain of coordinate-reference-system transforms, a nearest-neighbor distance computation per amenity type, an inverse-distance scoring step per amenity using the given weight and cutoff, and a final ranking step. Because this construction has no run-to-run randomness, it serves as the fixed reference point for the LLM arm rather than as a second experimental condition.

4.2 LLM plan construction

The LLM arm receives only the natural-language question — it is never given the weights, distance cutoffs, or even the fact that a rule-based reference exists — and is asked to produce an executable query plan (`QuerySpec`) using `LLMQuerySpecGenerator`, which is executed against the same underlying data. This was run $N = 20$ times at temperature 0.1, logging, for every run: the generated `QuerySpec` (operations, and each operation’s parameters), whether generation succeeded, whether execution succeeded, the resulting ranking, whether that ranking was degenerate (e.g., all-zero or all-tied scores indicating a silent failure rather than a real result), and wall-clock latency. Raw logs for all 20 runs are committed to the repository under `results/llm_runs/`.

Reaching a clean $N = 20$ batch required first finding and fixing two validation gaps in the underlying `smart_spatial_system` framework that were causing systematic, silent failures rather than genuine model unreliability; this is reported as a finding in its own right in Section 6.1, and the fixed release (`smart-spatial-system==0.2.4`) is the one pinned in this repository’s `requirements.txt`.

4.3 Comparison metric

The two arms are compared on three layers, computed in notebooks/ 04_comparison_metric.ipynb and defined precisely in [comparison_metric.md](#):

Layer 1 — Plan Agreement Rate (PAR). The structural agreement between runs: the fraction of runs (within an arm) whose sequence of operation *names* (ignoring parameter values and cosmetic variable naming) matches the arm’s most common (modal) sequence. PAR = 1.0 means every run produced the same shape of plan.

Layer 2 — Parametric agreement. For each amenity, the mean and standard deviation, across runs, of the weight and maximum-distance value the plan assigned to it. A standard deviation of 0 means every run chose the identical value; this layer is reported per amenity, not collapsed into a single number, because the interesting result (Section 5.3) is a difference in *which* values were chosen, not just how consistently.

Layer 3 — Rank Stability. The pairwise Spearman rank correlation (ρ , via `scipy.stats.spearmanr`) between every pair of runs’ final district rankings *within* an arm, plus ρ between each LLM run and the rule-based reference ranking. $\rho = 1.0$ across all pairs means every run produced the identical ranking.

Reliability metrics — generation/execution success rate, the rate of non-degenerate rankings, and wall-clock latency — are reported alongside these three layers but are not part of them; they measure whether a run produced *a* usable answer at all, which is a precondition for the three layers to be meaningful.

5. Results

5.1 Reliability

All 20 LLM runs succeeded at both generation and execution, and none produced a degenerate ranking (success rate = 1.0, non-degenerate rate = 1.0). Median wall-clock latency was 20.01 s (IQR 18.78–21.70 s).

5.2 Structural and outcome agreement (Layers 1 and 3)

Arm	N	PAR	Rank Stability (within-arm)	ρ vs. rule-based	Success rate	Median latency
Rule-based	5	1.0	1.0	—	1.0	—
LLM	20	1.0	1.0	0.9990	1.0	20.01 s

All 20 LLM runs produced the identical 9-operation sequence (coordinate transforms, chained nearest-neighbor distance computations to each amenity, scoring, ranking), so PAR = 1.0 exactly. All 190 pairwise Spearman correlations among the 20 LLM rankings equal 1.0 (rank stability = 1.0, sd = 0): every run’s final ranking was numerically identical to every other run’s, not merely similarly ordered.

Agreement between the LLM arm and the deterministic rule-based reference is likewise very high but not exactly 1.0 ($\rho = 0.998966$). Section 5.4 traces this to

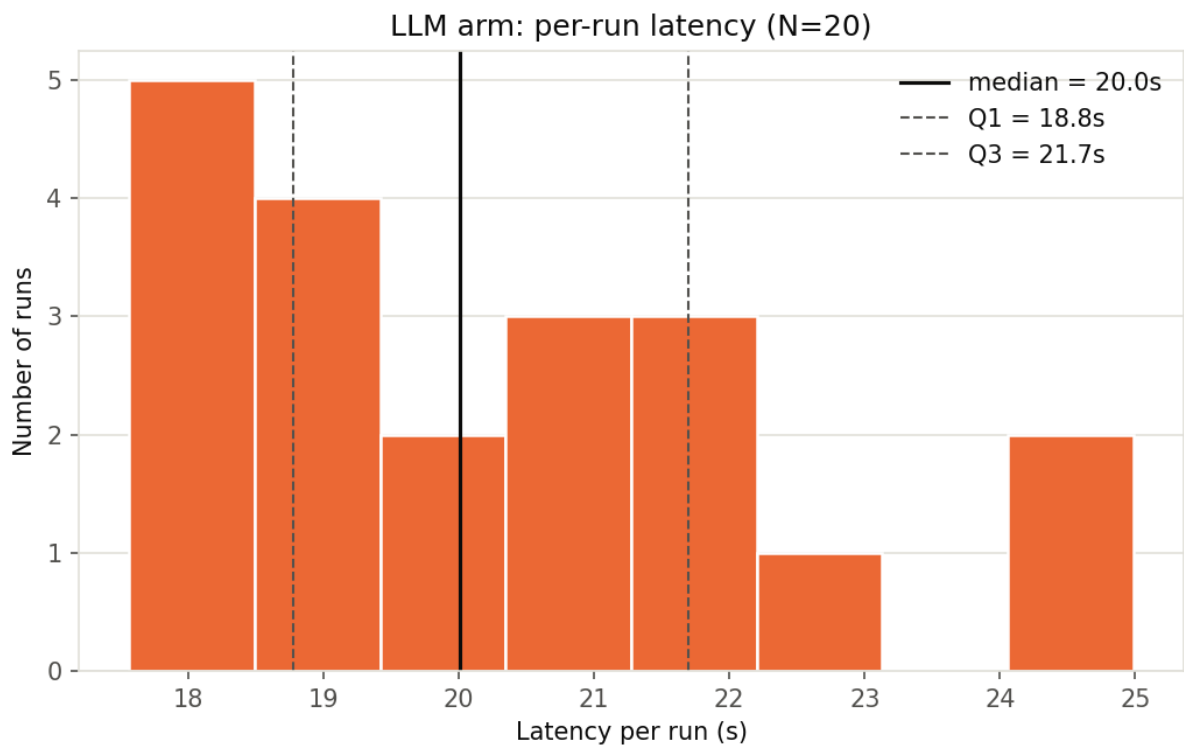


Figure 1: Distribution of the 20 LLM arm runs' end-to-end latency (question in, ranked output out). Latency clusters tightly around the median with no long tail, consistent with the same 9-operation plan running every time. Produced by notebooks/05_results.ipynb; saved as results/figures/fig3_latency.png.

a rounding artifact in the rule-based arm’s stored output rather than a genuine disagreement in the underlying distances. The figure below shows the resulting ranking as a choropleth (the previous section’s figure showed the LLM arm’s latency distribution).

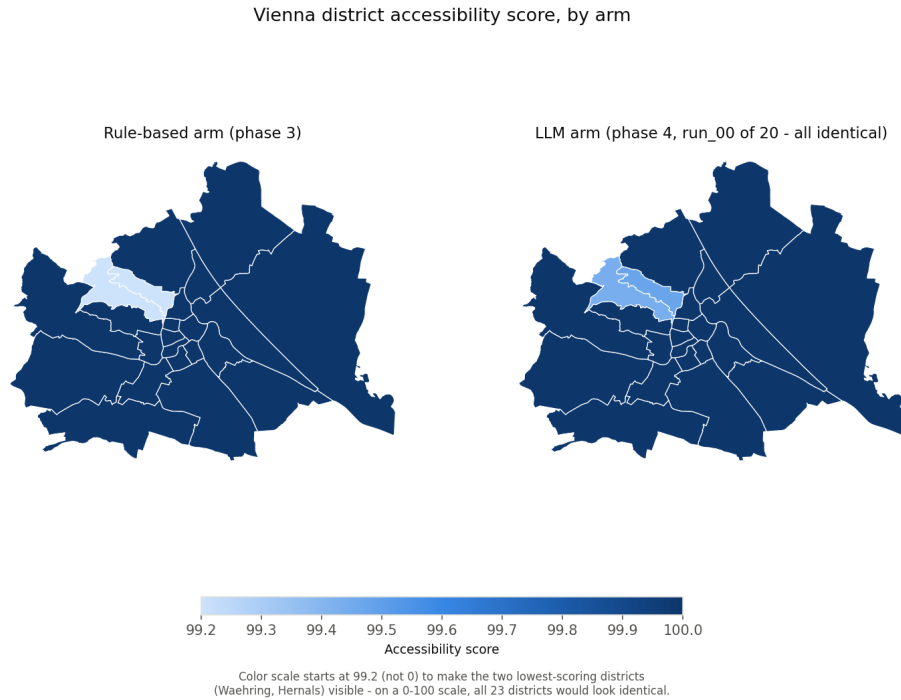


Figure 2: Vienna’s 23 districts shaded by accessibility score, rule-based arm (left) vs. one representative LLM arm run (right). The two maps are visually indistinguishable — both single out the same two lowest-scoring districts, Währing and Hernals — illustrating the near-perfect outcome agreement reported above. The color scale starts at 99.2, not 0, because on a full 0-100 scale all 23 districts would look identical (Section 7.1). Produced by notebooks/05_results.ipynb; saved as results/figures/fig1_choropleth.png.

5.3 Parametric agreement (Layer 2): reliability is not the same as judgment

All 20 LLM runs invented the *same* flat weighting scheme across amenity types — weight = 1 and max_distance = 800 m for metro, schools, and parks alike, with standard deviation 0.0 across every field in every run. The rule-based arm, in contrast, was *given* a deliberately unequal scheme (Section 3.2): metro weight 3.0/cutoff 800 m, schools weight 2.0/cutoff 1,000 m, parks weight 1.0/cutoff 1,500 m.

This is not an LLM error: the natural-language question never states these numbers, by design (Section 4.2), so there is no “correct” value for the model to recover — a flat default is a reasonable way to fill a genuinely under-specified gap. It does mean that perfect run-to-run *reliability* (Layer 1 and Layer 3, Section 5.2) is a separate property from matching a domain expert’s *judgment* about relative amenity importance (Layer 2): the LLM arm has the former in full and does not attempt the latter, because it was never given the information needed to attempt it.

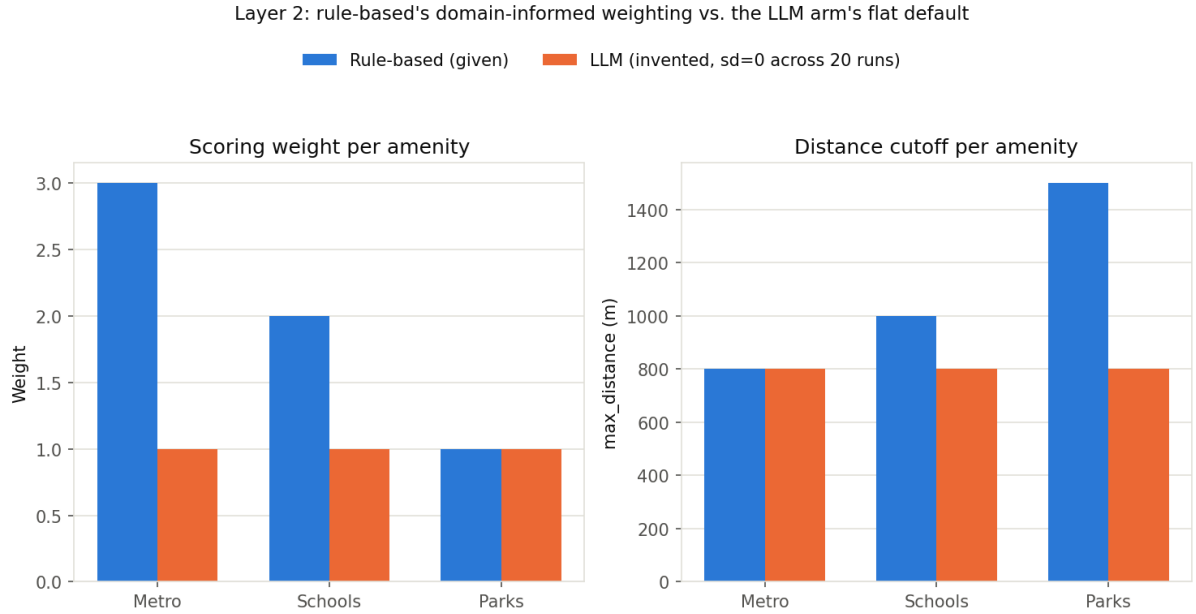


Figure 3: The rule-based arm’s given weighting (blue) vs. the LLM arm’s invented weighting (orange, identical across all 20 runs). Left: relative weight per amenity. Right: distance cutoff per amenity. The rule-based arm’s weighting favors metro over schools over parks, by design; the LLM arm defaults to treating all three equally. Produced by notebooks/05_results.ipynb; saved as results/figures/fig2_weights.png.

5.4 Why the two arms still nearly agree, and where the small gap comes from

Despite the different weighting schemes, the two arms’ rankings agree almost perfectly ($\rho = 0.998966$) because 21 of Vienna’s 23 districts are tied at the maximum accessibility score under *both* weighting schemes — the weighting only has room to matter where the ranking isn’t already saturated (Section 7.1). The two districts where it could matter, Währing and Hernals, are exactly where the small ρ gap originates, and tracing it down shows it is an artifact of value precision rather than a real disagreement: results/rule_based_ranking.csv stores both districts’ scores rounded to 99.2, which ties them under Spearman’s average-rank handling of ties, while the LLM arm’s higher-precision scores (99.472262 for Währing vs. 99.436835 for Hernals in run_00, computed from the same underlying 12.67 m / 13.52 m metro distances the rule-based arm also uses) rank them 22nd and 23rd distinctly. The two arms’ underlying geometry agrees; only the display precision of one arm’s stored output does not.

5.5 Robustness pilot: does this generalize beyond one model and one phrasing?

Everything above holds one model (gpt-4o-mini) and one exact phrasing fixed. To check whether the perfect reliability reported in Section 5.2 is a property of that specific pairing rather than something more general, we ran a smaller follow-up pilot — $N = 5$ per condition rather than $N = 20$, escalating only if a condition showed any deviation — along two independent axes: alternative models answering the identical question, and alternative phrasings of the same request answered

by the baseline model. Additional models were reached through AvalAI’s OpenAI-compatible endpoint, the same access path `smart_spatial_system`’s LLM client already supports.

The pilot surfaced a real, informative failure before it surfaced a clean result. `gpt-4o` — a stronger model in the same vendor family as the baseline — generated a structurally identical 9-operation plan on all 5 attempts but failed execution on all 5, because it named the ranking operation’s field parameter `ranking_field` instead of the framework’s actual `rank_field`. Checked directly against `smart_spatial_system`’s own source, this was a genuine parameter-name mismatch, not a transient error, and it traced to the same root cause as Section 6.1’s finding: the harness’s own schema guidance (`SYSTEM_HINTS`) documents the `score_features` operation’s schema in exhaustive detail but never documents `rank_features`’s parameter names, because the baseline model never happened to need that spelled out. This is exactly the caution raised by recent work on cross-model agent comparisons — that an apparent reliability difference between models is often really a difference in how completely the surrounding harness specifies the task, not a capability gap in the model itself ([Stop Comparing LLM Agents Without Disclosing the Harness](#)). Rather than retroactively editing the hints already quoted verbatim in Section 4.2 (which would undermine that section’s own citation of the exact text the $N = 20$ result ran against), we defined a second, explicitly extended hint string for this pilot only — phase 4’s hints plus one added requirement naming `rank_field` correctly — and re-ran every pilot condition under it.

Under the extended hints, every condition that successfully reached a model came back completely clean, at $N = 5$ each, independently recomputed from each condition’s raw run logs rather than taken from the pilot’s own summary: `gpt-4o`; two further models reached through AvalAI (`grok-3-mini-fast-beta` and `grok-3-fast`, a third vendor family); and all four prompt paraphrases (casual, formal, reordered-emphasis, and one with an added “relocation report” distractor detail). Every one of these seven conditions scored $PAR = 1.0$, $Rank\ Stability = 1.0$, and ρ against the rule-based reference = 0.998966 — the identical value reported in Section 5.2, for the identical reason given in Section 5.4. Two of the four prompt phrasings (formal, distractor) had in fact failed under the original hints for a *different* reason than `gpt-4o`’s: the model’s plan added an unrequested report/PDF-generation step, apparently triggered by report-adjacent wording, matching what recent work on paraphrase sensitivity predicts can happen to an LLM’s structured output under semantically equivalent rewordings ([Paraphrase-Induced Output-Mode Collapse: When LLMs Break Character Under Semantically Equivalent Inputs](#)). Both came back clean once the extended hints explicitly said not to add a report step unless one was requested — a second small, concrete illustration of a wording-sensitive plan-structure change, resolved the same way the model-side gap was.

Two of the pilot’s nine intended conditions were never evaluated successfully, despite three rounds of substitution, and are reported here as a closed, accepted gap rather than left open for further guessing (no further AvalAI model ids remained to try). The slot intended for a weak/legacy model (tried in turn as `gpt-3.5-turbo`, then `qwen3-235b-a22b-fp8-tput`) failed every single time for reasons unrelated to model behavior: the first id was rejected as deprecated, the second returned a generic AvalAI “invalid request” error on all 5 attempts. The slot intended for a second frontier model from a different vendor family (tried as `gemini-2.0-flash`, then `qwen3.8-2.4t-a95b`) produced a more informative failure on its second attempt: 3

of 5 calls timed out, but the other 2 of 5 *did* generate a plan — with the identical 9-operation structure as every other condition — and then failed execution with `score_features()` got an unexpected keyword argument `'score_field'`. Checked directly against the framework’s source, `score_features` has no such parameter (it takes `output_field` instead); `score_field` is only valid on `rank_features`. This model had applied the very parameter name Section 5.5’s extended hints added for `rank_features` to the *other* operation instead — a third, independent instance of the same harness-completeness lesson, this time showing that closing one model’s gap in the shared hints can open a different, model-specific gap for another model, rather than closing the question for every model at once. **We were not able to include a genuinely small or weaker model in this pilot** — every model that successfully returned a usable plan was a current-generation, broadly capable commercial model. This is an explicit, named gap (Section 7.4), not a result: the pilot’s clean finding should be read as *reliability generalized across the capable models this pilot could actually reach*, not as *reliability holds regardless of model capability*.

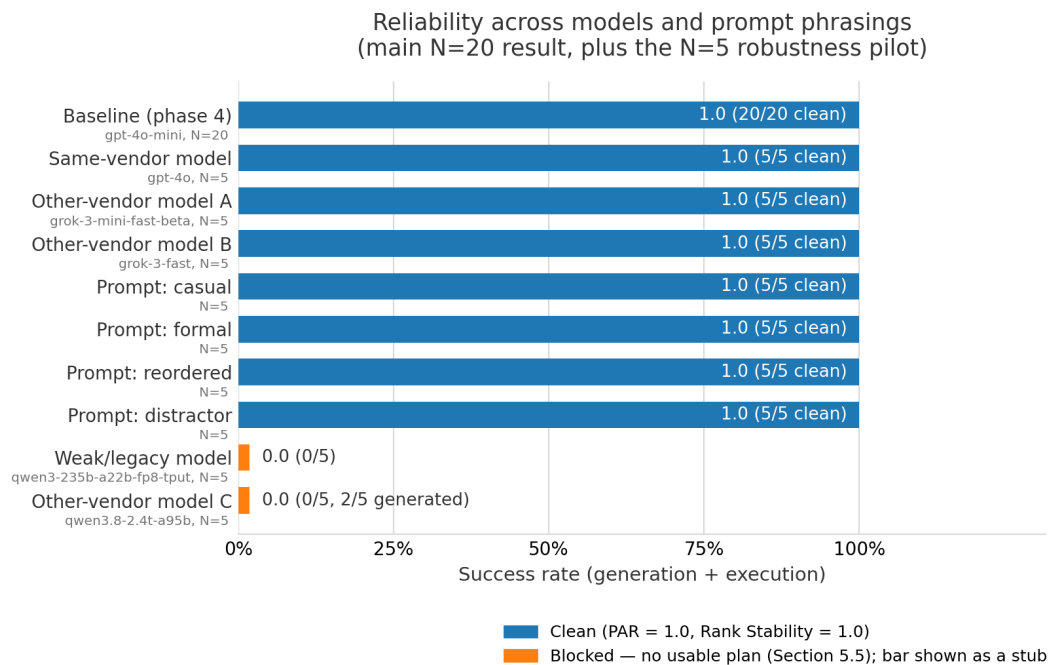


Figure 4: Success rate (generation and execution both succeeding) for the main N=20 baseline and each N=5 pilot condition. Seven of nine pilot conditions matched the baseline exactly (PAR = 1.0, Rank Stability = 1.0); the two blocked conditions (orange) never produced a usable ranking despite three rounds of model-id substitution and are reported as a closed, accepted limitation rather than left open (Section 7.4). Produced from `results/robustness_pilot/metrics_pilot.csv`; saved as `results/figures/fig4_pilot_reliability.png`.

6. Discussion

6.1 A paired finding: failure mode, then resolution

The clean, perfectly-reliable $N = 20$ batch reported in Section 5.2 is the end state of a longer process, and that process is itself worth reporting rather than editing out. Earlier batches of 20 LLM runs came back entirely degenerate — every run executed without raising an error, yet produced a meaningless ranking. Tracing this down (documented in full in the project’s phase-4 working notes) found two distinct root causes inside the underlying `smart_spatial_system` framework, not in the model’s plan itself:

1. A CRS-symmetry validator that falsely rejected every correctly-chained multi-hop distance plan of exactly the shape this case study’s own accessibility query produces (site $\rightarrow$ metro-distance $\rightarrow$ school-distance $\rightarrow$ park-distance), because it checked only the immediately preceding operation’s coordinate reference system rather than walking back through the full chain to find the originating transform.
2. A scoring-factor default that silently treated a distance field as boolean whenever the LLM’s generated plan omitted an explicit type, rather than raising an error — collapsing every district’s score to 0.0 with nothing in the pipeline signaling that anything had gone wrong.

Both were genuine defects in the framework’s validation logic, not limitations of the model: once fixed (released upstream as `smart-spatial-system==0.2.4`) and required rather than silently defaulted, the *same* LLM, at the *same* temperature, answering the *same* question, went from 20/20 degenerate to 20/20 clean, structurally identical, and outcome-identical. The lesson we draw is that an LLM-backed planner’s measured “reliability” is only as meaningful as the validation of the framework executing its plans; a framework that fails silently will make a perfectly capable model look unreliable, and a framework that fails loudly turns every future silent failure into a fixable bug report instead of an invisible wrong answer. This is consistent with the harness-engineering literature cited in Section 2, which frames exactly this kind of validation tightening as the mechanism by which agentic execution becomes predictable.

This same lesson recurred, independently, in the robustness pilot (Section 5.5): a stronger model in the same vendor family as the baseline initially failed 5/5 executions for a reason that turned out to be an incomplete schema hint, not a capability gap, and two prompt phrasings initially failed for a related but distinct hint-completeness reason. That it recurred on the very first attempt to extend this case study beyond its original single model and phrasing is, if anything, a stronger version of the same point: harness completeness is not a one-time fix particular to the baseline condition, but a moving target every new model or phrasing can expose a fresh gap in, and needs to be checked for explicitly rather than assumed.

6.2 Reliability versus judgment

Section 5.3’s finding generalizes beyond this one case study: asking whether an LLM planner is *reliable* (does it do the same thing every time?) is a different question from asking whether it makes the same *choices* a domain expert would (does it weight the right things the right amount?). This case study’s LLM arm scores perfectly on the first question and was never in a position to attempt the second,

because the weighting information was deliberately withheld from it. A deployment that cares about the second question needs to either supply that information explicitly in the prompt or query, or accept that the model’s default judgment is exactly that — a default, not a domain-informed choice — and validate it accordingly.

6.3 Near-agreement despite different judgment

Section 5.4’s finding is a caution about reading a single aggregate agreement number ($p = 0.998966$) as evidence that a weighting choice “doesn’t matter”: here it is high largely because the underlying data (Section 7.1) leaves the weighting little room to matter for 21 of 23 districts, not because the two weighting schemes are actually similar. A dataset with more separation across the full ranking would likely show a larger gap between the two arms’ rankings, meaning this particular near-agreement result should not be over-generalized past this specific, densely-served city.

7. Limitations

7.1 A ceiling effect in the rule-based ranking

Twenty-one of Vienna’s 23 districts tie at the maximum accessibility score (100.0) under the rule-based scoring. This was checked directly against the possibility that it was a bug — both the shapely-backed and pure-Python fallback paths in `smart_spatial_system`’s own distance calculation were read directly and confirmed to correctly return 0.0 when a point lies inside a polygon (standard, not a shortcut) — and against an independent point-in-polygon check written from scratch with no shared code with the library under test, run directly against the processed GeoJSON. The result: every one of the 23 districts contains at least one school and one park (211 schools and 1,063 parks, averaging roughly 9 and 46 per district respectively), so the distance-to-school and distance-to-park terms are genuinely 0.0 everywhere; exactly two districts, Währing and Hernals, contain none of the 109 metro stations, which is the only source of variation in the current scoring. This is not a bug — it is a real consequence of amenity density in a compact, well-served city — but it is a genuine limitation of this particular scoring design: a polygon-to-point “0 if any instance falls inside the boundary” distance saturates almost immediately in a city this dense, and a centroid- or population-weighted distance measure would differentiate the top of the ranking far more than this framing does. Readers should treat the specific rank ordering among the 21 tied districts as an artifact of the scoring choice, not as a finding about their relative accessibility.

7.2 Scope

This case study covers one city, one fixed natural-language question, one LLM at one temperature setting (0.1), and one comparison metric. It demonstrates a method and reports what that method found for this specific configuration; it is not a claim about LLM query planners in general, about this particular model’s behavior at other temperatures, or about other cities’ amenity distributions producing the same near-perfect agreement reported in Section 5.4.

7.3 $N = 20$ and further runs

Because all 20 LLM runs came back structurally and numerically identical (Section 5.2), a larger N would, for this specific configuration, mostly spend additional API budget confirming the same zero-variance result rather than revealing new variance. $N = 20$ was sufficient to characterize this setup; revisiting N upward is worthwhile only alongside a change that could plausibly reintroduce variance, such as the temperature sweep discussed next.

7.4 The robustness pilot’s own limits

The pilot in Section 5.5 is deliberately smaller than the main result and should not be read as carrying the same weight. Each condition ran at $N = 5$, not $N = 20$; a 5-run pilot has real limits on its power to detect low-frequency variance, so “clean at $N = 5$ ” is reported as *stable, not yet confirmed at $N = 20$* rather than as an equally strong claim to the main result. The three additional models that did return usable plans (gpt-4o, grok-3-mini-fast-beta, grok-3-fast) are all current-generation, broadly capable commercial models; every attempt to include a weaker, older, or open-weight model in the two remaining slots failed, across three rounds of substitution — most for infrastructure reasons (a deprecated or unrecognized model id, a rejected request, or a request timeout), and one for the model-specific parameter-confusion bug described in Section 5.5 — and this is reported as a closed, accepted gap rather than left open pending a fourth substitution, since no further model ids remained to try. The pilot therefore cannot speak to whether perfect reliability holds for weaker models — that remains open, and is the most direct way this specific finding could still be overturned by a follow-up.

8. Conclusion and Future Work

For this case study’s specific configuration — one accessibility question, one city, one model, temperature 0.1 — an LLM-backed spatial query planner, once the underlying framework’s validation gaps were closed, was exactly as reliable across 20 repeated runs as a hand-written rule-based planner ($PAR = 1.0$, Rank Stability = 1.0), and its output agreed with the deterministic reference almost exactly ($\rho = 0.998966$), with the residual gap traced to a display-precision artifact rather than a real disagreement. The one substantive difference between the two arms was a matter of judgment, not reliability: the LLM defaulted to a flat weighting across amenity types where the rule-based arm was given a deliberately unequal one, and the case study’s data happened to make that difference nearly invisible in the final ranking. A smaller follow-up pilot (Section 5.5) found this same perfect reliability recurring across two further models and four independent phrasings of the question, once a second harness gap specific to those conditions was closed the same way the original gaps were — a second, independent illustration of Section 6.1’s central lesson, and a reason for cautious optimism that the main result is not an artifact of one particular model or wording, within the capable-commercial-model range the pilot could actually test (Section 7.4).

Four directions follow from the limitations above. First, a temperature sweep beyond the single 0.1 value used here, now that a clean, zero-variance baseline exists to compare against, would test whether the perfect reliability reported here is a property of low temperature specifically. Second, a population- or centroid-weighted distance measure would relieve the ceiling effect described in Section

7.1 and likely produce a more discriminating ranking against which to re-run this same comparison. Third, supplying the amenity weights explicitly in the natural-language query (rather than withholding them, as this case study deliberately does) would let Layer 2 test whether the LLM can *recover* a specified domain judgment, as distinct from inventing a reasonable default for an unspecified one. Fourth, and most directly motivated by Section 7.4, escalating the pilot’s successful conditions to $N = 20$ and specifically pursuing a smaller or open-weight model through a provider that keeps such models available would close the one gap this case study’s own robustness check could not.

9. Reproducibility and Data & Code Availability

All code, data, run logs, notebooks, and figures for this case study are released in the [smart-spatial-vienna-accessibility](#) repository under the MIT license. Concretely, the repository contains: `data/processed/` — the cleaned OpenStreetMap layers (district boundaries, metro stations, schools, parks) both arms query; `notebooks/01-06` — one notebook per phase, from data acquisition through the robustness pilot, each runnable end to end; `results/llm_runs/` and `results/robustness_pilot/` — every individual run’s raw JSON record (the generated plan, success/failure, the resulting ranking, latency), not just an aggregate; `results/metrics.csv` and `results/robustness_pilot/metrics_pilot.csv` — the computed PAR / Rank Stability / ρ / success-rate numbers this paper reports, recomputed directly from those raw logs; `results/figures/` — Figures 1–4 as reproduced in this paper; and `paper/paper.md` (this document) alongside `paper/PLAN.md`, which records the project’s phase-by-phase decisions and findings as they were made. The repository pins `smart-spatial-system==0.2.4` (the fixed release discussed in Section 6.1); phases 2, 3, 5, and 6 (data exploration, the rule-based arm, the comparison metric, and the results/figures notebooks) run fully offline from the committed `data/processed/` and `results/llm_runs/` directories with no network access or API key required. Only phase 1 (initial data acquisition from OpenStreetMap) and phase 4 (the LLM arm itself) need network access, and only phase 4 needs an LLM API key. The repository’s README documents a zero-local-install path (a devcontainer-based cloud execution option) for readers who want to reproduce or extend this analysis without setting up a local Python environment.

Raw amenity and boundary data are from OpenStreetMap, © OpenStreetMap contributors, available under the Open Database License (ODbL); see `data/README.md` for exact queries, license terms, and attribution.

If you use this case study, please cite both this repository and [Smart Spatial System](#).

References

- Vich, G. et al. *A city of cities: Measuring how 15-minutes urban accessibility shapes human mobility in Barcelona*. [arXiv:2103.15638](#) / [PLOS ONE](#).
- *Measuring 15-minute city access and inequality*. Congress for the New Urbanism, Public Square. [Link](#).
- *Just around the corner: Accessibility by proximity in the 15-minute city*. ScienceDirect. [Link](#).

- *How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines.* [arXiv:2605.28840](#).
- *Harness Engineering for Predictable Agentic Systems: An Empirical Study of Deterministic Execution Constraints.* [arXiv:2608.26197](#).
- *Evaluation and Benchmarking of LLM Agents: A Survey.* [arXiv:2507.21504](#).
- *Stop Comparing LLM Agents Without Disclosing the Harness.* [arXiv:2605.23950](#).
- *Paraphrase-Induced Output-Mode Collapse: When LLMs Break Character Under Semantically Equivalent Inputs.* [arXiv:2605.04665](#).